

VIRUS KICKER 2

Voilà de quoi soigner vos disquettes (pas vos jeux originaux). On installera le programme dans le boot-secteur d'une disquette vierge, grâce au 3ème listing. Pas trop recommandé aux novices.

JEDI & ULTIMATUM (from The Heavy Killers)

LE LISTING ASSEMBLEUR

```
* Viruskicker 2
* par: - Jedi of Sector One
* - Ultimatum of Zap Creation
* from the Heavy Killers
```

```
* Viruskicker 2
* par: - Jedi of Sector One
* - Ultimatum of Zap Creation
* from the Heavy Killers
```

```
start: pea thk(pc)
move $9, -(sp)
trap #1
lea 2(sp), a5
lea pat(pc), a0
```

```
check: move $1420, w, d0
move $1, d0, a6
cmpi $12123456, (a6)
bne.s virus
```

```
virus: move $1 a0, (a5)
trap #1
lea menu(pc), a0
```

```
move $1 a0, (a5)
trap #1
subq w #2, (sp)
```

```
key: trap #1
swap d0
cmpi $143b, d0
```

```
move $1, (sp)
clrl $1, -(sp)
lea 8(w), a1
```

```
lea retout(pc), a0
moveq #9, d7
move $1 (a0), (a1)+
```

```
retout: jmp $12 w
move $1, a w, a0
jmp (a0)
```

```
lea nan(pc), a0
moveq #0, d0
move $1 d0, d1
```

```
IF cc=255 AND page=1
cc=255
ENDIF
```

```
LOCK
a5
PRINT CHR$(27); "p"
checksum
```

```
PRINT CHR$(27); "q"
RETURN
```

```
PROCEDURE trait erreur
err=MIDS (ERR$(ERR$(1)), 5)
err=MIDS (err$, 1, LEN(err)-9) "
```

```
err=MIDS (err$, SUCC(2*ABS(MIDS (err$, 1, 1))+""))
IF INSTR(err$, "I")
```

```
err=MIDS (err$, 1, INSTR(err$, "I")-1) "+" "MIDS (err$
, SUCC(INSTR(err$, "I"))
ENDIF
```

```
ALERT 0, err$, 1, " OK ", "tst"
RETURN
```

```
PROCEDURE catalog
n disks= " NOUVEAU"
CLR nbf, nbf, c
```

```
detail: directory
GRAPHMODE 1
DEFPILL 0
```

```
PROC 0, 24, 639, 199
COLOR 1
BOX 0, 28, 639, 122
```

```
BOX 100, 122, 639, 198
PRINT AT(12, 17); "TICHER"
PRINT AT(12, 18); "LONGUEUR" "
```

```
PRINT AT(12, 20); "TYPE"
PRINT AT(12, 21); "GENRE"
PRINT AT(12, 22); "EPIQUE"
PRINT AT(12, 23); "OBJET"
PRINT AT(15, 17); "NOM DU DISQUE: "n disks
```

```
move $1 d1, d2
move $1 d2, d3
move $1 d3, d4
move $1 d4, d5
move $1 d5, d6
move $1 d6, d7
```

```
killall: move $1 d0-d7, (a0)
add $120, a0
bra.s killall
```

```
nan1: cmpi $143b, d0
bne.s nan2
bchg #1, $1, (a0)
bne.s nan3
```

```
addq $1 #6, sp
rta
nan2: cmpi $143b, d0
bne.s nan3
```

```
lea cur(pc), a6
move $1 a6, (a5)
addq #2, (sp)
```

```
trap #1
lea label(pc), a6
move $1 a6, (a5)
```

```
addq #1, (sp)
trap #1
move $12020, (a6)
```

```
addq $1 #6, sp
youpi
nan3: cmpi $143b, d0
rtn
```

```
move $1 32000 (a6)
move $1, -(sp)
trap #14
```

```
move $1 32008 (a6), d1-d5
lea start-28 (pc), a6
move $1, (sp)
```

```
move $1, (sp)
move $1 a6, (sp)
move $1254, d1
add (a6)+, d0
```

```
PRINT AT(16, 23); MIDS ("Mon cacheCaché ", VAL
(MIDS (res$, 4, 1)) * 91, 9)
```

```
key=INDEXS
b=ASC (RIGHTS (key$))
c=ASC (LEFTS (key$))
```

```
IF (b=80 OR b=72) AND c=0
PRINT AT(2, c*4); " "MIDS (cat$(cur), 4) +
SPACES(30)
```

```
ENDIF
IF b=80 AND c=0 AND cur=shf ! Descendre
INC cur
```

```
IF c=1
INC cy
ELSE
INC cy
```

```
MOVE X105 (3) + 40 * 160, X105 (3) + 32 * 160,
80 * 160
ENDIF
```

```
IF b=72 AND c=0 AND cur=1 ! Monter
DEC cur
```

```
IF c=1
DEC cy
ELSE
DEC cy
```

```
MOVE X105 (3) + 32 * 160, X105 (3) + 40 * 160,
80 * 160
ENDIF
```

```
EXIT IF key=CHR$(13) OR key=CHR$(27)
LOOP
PROC 0, 24, 639, 199
```

```
a5
info gen
RETURN
```

```
PROCEDURE detail directory
a5
DO UNTIL test
```

```
IF LEFTS (CUR (FGETOAT (1) + 30)) < " "
IF BYTES (FGETOAT (1) + 21) AND 0
n disks=LEFTS (CUR (FGETOAT (1) + 30), 60)
```

```
INC nbf
cat$(nbf)=LEFTS ("A:" + "path$(CUR (
FGETOAT (1) + 30), 60)
rps (nbf)=CUR (BYTES (FGETOAT (1) + 21)) + MIDS
(FGETOAT (1) + 66)
```

```
ENDIF
IF BYTES (FGETOAT (1) + 21) AND 16
INC nbf
```

```
path$(nbf)=CUR (FGETOAT (1) + 30) + " "
detail: directory
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
WHILE path$(nbf) < path$, RINSTR (path$, " ",
LEN (path$) - 1) + " " + CUR (FGETOAT (1) + 30) + " "
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

```
test=SFIRST ("A:" + "path$(CUR (FGETOAT (1) + 30), 60)
path$, " ", LEN (path$) - 1) + " ", -1)
```

68000 & GFA



ANTI-VIRUS
Toutes Résolutions

Pi80

```
DATA 2485457, 4941409, D63C0C, 1A0208E, 52574681
DATA 30C2020, 5C84000, FFA0C00, 3C4600, F72278, 4E3B3C
DATA 1A2A, 4878001, 42A746E, 7003F3C, 8484E, 4C8D03E
DATA 7004D0A, F2F848E, 360006, 3EBC009, 2F4E002
DATA 32C00F, 0055135, F7F323E, 1234540, 3C14E4E
DATA 4F0E018, 60A1B65, 186618, 431B5920, 2A46544
DATA 692F365, 6374672, 204F6E5, 2F54484B, 1B59230
DATA 7076573, 6567473, 1B59242D, 56697275, 7366963
DATA 6857220, 3200169, 2B202A5E, 4932553, 20414C5
DATA 52542407, 1B5928, 2F2046E, 20766972, 7532000
DATA 1B592C20, 4A69736B, 2086616D, 633A2020, 20202020
DATA 20202020, 20202020, 20202020, 20202020, 20202020
DATA 20202020, 4A631A, 52557465, 7400046, 323A34E
DATA 7078206A, 6932062, 6F6740D, A66333A, 5365742D
DATA 66616D65, D0A634, 3A53796E, 63D0A53
DATA 70616365, 3A53796E, 741B592C, 20D00000
```

LE LISTING GFA POUR INSTALLER
LE PROGRAMME EN BOOT-SECTEUR

```
OPEN "1", #1, "VIRUSKICKER.PRG"
SEEK #1, 28
a5=INSTR (A4, #1)
CLOSE #1
chr$(STRINGS (1024, 0)
a5=VARR (chr$(a5) AND 4FFFFFFF)
-ORIG (8, 1, chr$(a5), 0, 0, 1, 0, 0, 1)
MOVE VARR (a5), a5, a5, 28, 512
LPC: a5, 4131F0765
LPC: a5, 418274014
FOR a5=a5 to a5+508 STEP 2
ADD chr$(a5), a5
NEXT a5
LPC: a5, 418274014
-ORIG (8, 1, chr$(a5), 1, 0, 0, 1, 0, 0, 1)
```

```
FOR i=V.tamp$ TO V.tamp$+511 STEP 2
ADD check, DPEX (i)
NEXT i
```

```
PRINT AT(11, 3); "Check : ".HEX$(check, 4); " "
RETURN
```

```
PROCEDURE boot
stamp=tamp$
sect=1
pist=0
face=0
lface=0
```

```
FOR i=V.tamp$ TO V.tamp$+511 STEP 2
ADD check, DPEX (i)
NEXT i
```

```
IF HEX$(check, 4) = "1234" AND DPEX (V.tamp$) < 0
ALERT 2, "I BOOT EXECUTABLE (PEUT-ETRE UN VI
RUS ", 1, " OK ", "rep8
```

```
ELSE
ALERT 0, "PAS DE BOOT EXECUTABLE ", 1,
" OK ", "rep8
```

```
ENDIF
stamp=stamp$
RETURN
```

```
PROCEDURE info gen
PRINT AT(1, 22); "[1]: Lire secteur [7]: Texte
[0]: Quitter [M]: Mettre dans buffer [5]: Sortie buf-
fer"
```

```
PRINT AT(1, 23); "[8]: Ecrire secteur [H]: hexa
[H]: Recherche [D]: Disque [K]: Catalogue [B]: Boot"
```

```
PRINT AT(1, 24); "[C]: Checksum. Utiliser : "
```

```
OUT 5, 1
OUT 5, 2
OUT 5, 3
OUT 5, 4
```

```
PRINT " pour déplacer le curseur. ";
OUT 5, 1
OUT 5, 2
```

```
PRINT " pour choisir le secteur, ";
OUT 5, 3
OUT 5, 4
```

```
PRINT " pour choisir la piste, ";
PRINT AT(1, 24); "SPC(80);
PRINT AT(1, 24); " " [F] pour choisir la
face, [Esc] pour sortir " ; 14 espaces au début
PRINT AT(1, 1)
RETURN
```

Minitel
STMAGICIEL:

3615 STMAG